



EDUQA.PE

INTRODUCCIÓN A SQLITE CON PYTHON



SESIÓN 01

El motor, la conexión y las tablas

AUTOR

Alejandro Valentino Seminario Medina

AI Engineer



Qué es SQLite

SQLite es un motor de base de datos relacional que guarda toda la información en un solo archivo del disco. No hay un servidor al que conectarse ni un proceso que administrar.

Nota técnica. SQLite es una biblioteca enlazada dentro del proceso de la aplicación, no un servicio separado. La documentación oficial lo compara con `fopen()`, no con PostgreSQL: compete con leer y escribir archivos propios, no con un servidor de base de datos.

Documentación: [Qué es SQLite](#) · [Usos apropiados](#)



Por qué se usa desde Python

El módulo `sqlite3` forma parte de la biblioteca estándar de Python. No hay que instalar nada ni levantar ningún servicio: importar el módulo basta.

Nota técnica. El módulo implementa la especificación DB-API 2.0 descrita en el PEP 249, la misma interfaz que usan los conectores de PostgreSQL y MySQL. Lo que se aprende aquí se traslada casi sin cambios a esos motores.

```
import sqlite3

print(sqlite3.sqlite_version)
```



Por qué se usa desde Python

Salida

3.45.1

Documentación: [sqlite3](#)



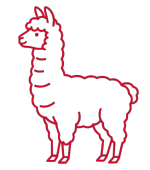
Conectarse a una base de datos

La función `connect()` recibe la ruta del archivo y devuelve un objeto de conexión. Si el archivo no existe, SQLite lo crea en la primera escritura.

Nota técnica. El archivo se crea de verdad recién cuando se escribe algo. Conectar a un nombre inexistente y cerrar sin crear tablas no deja ningún archivo en el disco.

```
conexion = sqlite3.connect("tienda.db")
```

Pasar la cadena `":memory:"` en lugar de una ruta crea una base que vive solo en la memoria y desaparece al cerrar la conexión. Es lo habitual en pruebas.



Conectarse a una base de datos

Nota técnica. Cada conexión a ':memory:' crea una base independiente. Dos conexiones en memoria dentro del mismo programa no comparten datos, salvo que se use el modo URI con caché compartida.

```
conexion = sqlite3.connect(":memory:")
```

Documentación: [sqlite3.connect\(\)](#)



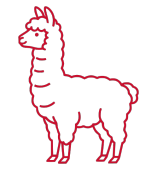
El cursor

El cursor es el objeto que envía las sentencias SQL y recorre los resultados. Se obtiene desde la conexión.

Nota técnica. Una conexión admite varios cursores a la vez. Cada uno mantiene su propia posición de lectura, por lo que dos consultas simultáneas no interfieren entre sí.

```
cursor = conexion.cursor()
```

El método **execute()** envía una sentencia al motor.



El cursor

```
cursor.execute("SELECT sqlite_version()")  
print(cursor.fetchone())
```

Salida

('3.45.1' ,)

Documentación: [Cursor](#) · [Cursor.execute\(\)](#) · [Cursor.fetchone\(\)](#)



Tipos de datos

SQLite tiene exactamente cinco clases de almacenamiento: **NULL**, **INTEGER**, **REAL**, **TEXT** y **BLOB**. No hay más. Cada una se recorre a continuación con su equivalente en Python.

Nota técnica. La función `typeof()` del motor devuelve el nombre de la clase de almacenamiento del valor, no el tipo declarado de la columna. Es la herramienta para comprobar qué guardó SQLite realmente.

Para los ejemplos se usa una tabla con una sola columna sin tipo declarado, de forma que cada valor conserve su clase original.

```
cursor.execute("CREATE TABLE tipos (valor)")
```



Tipos de datos

Documentación: [Tipos de datos](#) · [Adaptar tipos de Python](#) · [Afinidad de tipo](#)



NULL

Representa la ausencia de valor. En Python le corresponde **None**.

Nota técnica. NULL no es cero ni cadena vacía: significa que el dato no está. Esa distinción cambia el resultado de las comparaciones y de las funciones de agregación, y se trata en detalle en la sesión 3.

```
cursor.execute("INSERT INTO tipos VALUES (?)", (None,))
cursor.execute("SELECT valor, typeof(valor) FROM tipos")
print(cursor.fetchone())
```

Salida

(None, 'null')

Documentación: [Tipos de datos](#) · [Manejo de NULL](#)



INTEGER

Guarda números enteros con signo. En Python le corresponde **int**.

Nota técnica. SQLite almacena el entero en 1, 2, 3, 4, 6 u 8 bytes según su magnitud, y elige el tamaño solo. El rango máximo es el de 8 bytes con signo, de -9223372036854775808 a 9223372036854775807. Un int de Python mayor que eso provoca un OverflowError.

```
cursor.execute("DELETE FROM tipos")
cursor.execute("INSERT INTO tipos VALUES (?)", (2500,))
cursor.execute("SELECT valor, typeof(valor) FROM tipos")
print(cursor.fetchone())
```



INTEGER

Salida

(2500, 'integer')

Documentación: [Tipos de datos](#)



REAL

Guarda números con parte decimal. En Python le corresponde **float**.

Nota técnica. REAL es un número de punto flotante de 8 bytes según el estándar IEEE 754, el mismo formato que usa float en Python. Arrastra por tanto las mismas imprecisiones: $0.1 + 0.2$ no da exactamente 0.3. Para importes de dinero conviene guardar céntimos como INTEGER.

```
cursor.execute("DELETE FROM tipos")
cursor.execute("INSERT INTO tipos VALUES (?)", (2500.50,))
cursor.execute("SELECT valor, typeof(valor) FROM tipos")
print(cursor.fetchone())
```



REAL

Salida

(2500.5, 'real')

Documentación: [Tipos de datos](#)



TEXT

Guarda cadenas de caracteres. En Python le corresponde **str**.

Nota técnica. SQLite guarda el texto en la codificación de la base, que por omisión es UTF-8. El módulo `sqlite3` devuelve siempre un `str` ya decodificado, de modo que los acentos y la ñe no requieren ningún tratamiento adicional.

```
cursor.execute("DELETE FROM tipos")
cursor.execute("INSERT INTO tipos VALUES (?)", ("Ana Quispe",))
cursor.execute("SELECT valor, typeof(valor) FROM tipos")
print(cursor.fetchone())
```



TEXT

Salida

```
('Ana Quispe', 'text')
```

Documentación: [Tipos de datos](#)



BLOB

Guarda datos binarios tal cual, sin interpretarlos. En Python le corresponde **bytes**.

Nota técnica. BLOB significa binary large object. Sirve para imágenes, archivos o cualquier secuencia de bytes. La documentación oficial señala que guardar archivos pequeños, por debajo de unos 100 kB, resulta más rápido dentro de la base que como archivos sueltos en el disco.

```
cursor.execute("DELETE FROM tipos")
cursor.execute("INSERT INTO tipos VALUES (?)", (b"■PNG",))
cursor.execute("SELECT valor, typeof(valor) FROM tipos")
print(cursor.fetchone())
```



BLOB

Salida

`(b'PNG', 'blob')`

Documentación: [Tipos de datos](#)



Los tipos de Python que no tienen equivalente

Solo esos cinco tipos viajan de forma directa. Cualquier otro, como `datetime`, `Decimal` o `list`, produce un error salvo que se registre un adaptador.

Nota técnica. La correspondencia completa es `None` con `NULL`, `int` con `INTEGER`, `float` con `REAL`, `str` con `TEXT` y `bytes` con `BLOB`. Para el resto hay que convertir de forma explícita antes de insertar, o registrar un adaptador con `sqlite3.register_adapter()`.

```
from decimal import Decimal

cursor.execute("INSERT INTO tipos VALUES (?)", (Decimal("10.5"),))
```



Los tipos de Python que no tienen equivalente

Salida

```
sqlite3.ProgrammingError: Error binding parameter 1: type 'Decimal' is not supported
```

Documentación: [Adaptar tipos de Python](#) · [Excepciones de sqlite3](#)



Afinidad de tipo

El tipo declarado en la columna es una preferencia, llamada afinidad, no una restricción. Una columna declarada **INTEGER** puede terminar guardando texto.

Nota técnica. La afinidad se deduce del nombre declarado: si contiene INT, la afinidad es INTEGER; si contiene CHAR, CLOB o TEXT, es TEXT. Por eso VARCHAR(10) es válido, pero el número 10 no limita nada en absoluto.

```
cursor.execute("CREATE TABLE prueba (valor INTEGER)")
cursor.execute("INSERT INTO prueba VALUES ('hola')")
cursor.execute("SELECT valor, typeof(valor) FROM prueba")
print(cursor.fetchone())
```



Afinidad de tipo

Salida

`('hola', 'text')`

Documentación: [Afinidad de tipo](#) · [Peculiaridades de SQLite](#) · [Funciones básicas](#)



Tablas STRICT

Añadir la palabra **STRICT** al final de la sentencia **CREATE TABLE** obliga a que cada columna acepte solo su tipo declarado. Está disponible desde la versión 3.37 del motor.

Nota técnica. En una tabla STRICT los tipos permitidos se reducen a INT, INTEGER, REAL, TEXT, BLOB y ANY. Cualquier otro nombre, como VARCHAR, produce un error al crear la tabla.

```
cursor.execute("CREATE TABLE estricta (valor INTEGER) STRICT")
cursor.execute("INSERT INTO estricta VALUES ('hola')")
```

Salida

```
sqlite3.IntegrityError: cannot store TEXT value in INTEGER column estricta.valor
```



Tablas STRICT

Documentación: [Tablas STRICT](#) · [IntegrityError](#)



Crear una tabla

La sentencia **CREATE TABLE** define el nombre de la tabla y sus columnas. En Python se envía como una cadena, normalmente entre comillas triples para que ocupe varias líneas.

```
cursor.execute("""
    CREATE TABLE vendedores (
        id      INTEGER PRIMARY KEY,
        nombre TEXT    NOT NULL,
        region TEXT    NOT NULL
    )
""")
```



Crear una tabla

Nota técnica. Una columna declarada exactamente como INTEGER PRIMARY KEY no crea una columna nueva: se vuelve un alias del rowid interno que toda tabla ya tiene. Es el acceso más rápido posible a una fila.

Documentación: [CREATE TABLE](#)



Restricciones

Una restricción es una regla que el motor verifica en cada escritura. Son cuatro las de uso corriente, y cada una se recorre por separado a continuación.

Documentación: [Restricciones de columna](#)



NOT NULL

Obliga a que la columna tenga siempre un valor. Rechaza cualquier intento de guardar **NULL** en ella.

Nota técnica. Una columna sin NOT NULL admite NULL aunque nunca se le pase ese valor de forma explícita: si el INSERT no la menciona y no tiene DEFAULT, SQLite guarda NULL.

```
cursor.execute("CREATE TABLE t1 (nombre TEXT NOT NULL)")
try:
    cursor.execute("INSERT INTO t1 (nombre) VALUES (?)", (None,))
except sqlite3.IntegrityError as e:
    print(e)
```



NOT NULL

Salida

NOT NULL constraint failed: t1.nombre

Documentación: [Restricciones de columna](#)



UNIQUE

Impide que dos filas tengan el mismo valor en esa columna.

Nota técnica. SQLite implementa UNIQUE creando un índice único por debajo, de modo que la restricción acelera además las búsquedas por esa columna. Salvedad importante: varios NULL sí se admiten, porque dos valores ausentes no se consideran iguales entre sí.

```
cursor.execute("CREATE TABLE t2 (correo TEXT UNIQUE)")
cursor.execute("INSERT INTO t2 VALUES (?)", ("ana@correo.com",))
try:
    cursor.execute("INSERT INTO t2 VALUES (?)", ("ana@correo.com",))
except sqlite3.IntegrityError as e:
    print(e)
```



UNIQUE

Salida

UNIQUE constraint failed: t2.correo

Documentación: [Restricciones de columna](#)



DEFAULT

Indica el valor que se guarda cuando el **INSERT** no menciona esa columna.

Nota técnica. El valor por defecto debe ser una constante o una de las expresiones especiales CURRENT_TIME, CURRENT_DATE y CURRENT_TIMESTAMP. No puede referirse a otra columna ni llamar a una función arbitraria.

```
cursor.execute("CREATE TABLE t3 (nombre TEXT, activo INTEGER DEFAULT 1)")
cursor.execute("INSERT INTO t3 (nombre) VALUES (?)", ("Laptop",))
cursor.execute("SELECT nombre, activo FROM t3")
print(cursor.fetchone())
```

Salida

('Laptop', 1)



DEFAULT

Documentación: [Restricciones de columna](#)



CHECK

Verifica una condición sobre la fila antes de guardarla.

Nota técnica. CHECK se evalúa en cada INSERT y UPDATE. No puede consultar otras tablas ni usar subconsultas, porque debe poder verificarse con la fila a la vista. Si la condición da NULL, la restricción se considera cumplida.

```
cursor.execute("CREATE TABLE t4 (precio REAL CHECK (precio > 0))")
try:
    cursor.execute("INSERT INTO t4 VALUES (?)", (-100,))
except sqlite3.IntegrityError as e:
    print(e)
```



CHECK

Salida

```
CHECK constraint failed: precio > 0
```

Documentación: [Restricciones de columna](#)



Las cuatro juntas

Una tabla real combina varias restricciones en la misma definición.

```
cursor.execute("""
    CREATE TABLE productos (
        id      INTEGER PRIMARY KEY,
        nombre TEXT    NOT NULL UNIQUE,
        precio REAL    NOT NULL CHECK (precio > 0),
        activo  INTEGER NOT NULL DEFAULT 1
    )
""")
```



Las cuatro juntas

Al violar una restricción, el módulo lanza `sqlite3.IntegrityError`. Esa excepción hereda de `sqlite3.DatabaseError`.

Nota técnica. La jerarquía completa está definida por el PEP 249: Warning y Error cuelgan de Exception, y de Error descienden InterfaceError y DatabaseError. IntegrityError cubre las violaciones de restricción, incluidas UNIQUE, NOT NULL, CHECK y clave foránea.

```
try:
    cursor.execute("INSERT INTO productos (nombre, precio) VALUES ('Laptop', -100)")
except sqlite3.IntegrityError as e:
    print("Rechazado:", e)
```



Las cuatro juntas

Salida

Rechazado: CHECK constraint failed: precio > 0

Documentación: [Restricciones de columna](#) · [Excepciones de sqlite3](#) · [IntegrityError](#)



Sobre AUTOINCREMENT

La palabra **AUTOINCREMENT** casi nunca hace falta. Sin ella, SQLite ya asigna identificadores solo a una columna **INTEGER PRIMARY KEY**.

Nota técnica. La única diferencia es que AUTOINCREMENT garantiza que un identificador nunca se reutilice tras borrar filas. A cambio mantiene una tabla interna llamada `sqlite_sequence` y añade una escritura por inserción. La documentación oficial recomienda evitarla salvo que esa garantía sea necesaria.

Documentación: **AUTOINCREMENT** · **Tablas con ROWID**



Fin de la sesión

El motor, la conexión y las tablas

Alejandro Valentino Seminario Medina